Borla

Software Requirements Specification

Student: **George Boadu Junior**

Student ID: **22427354**

Course: CSCD 602 — Advanced Software Engineering, University of Ghana

Document: SRS.pdf · Version 1.0 · 13 August 2026

1. Introduction

1.1 Purpose

This document specifies the requirements for **Borla**, a real-time matchmaking application connecting Ghanaian households that have waste to dispose of with nearby roaming waste collectors. It is the Software Requirements Specification (SRS) deliverable for the CSCD 602 Advanced Software Engineering individual capstone examination.

1.2 Scope

Borla is deliberately a **matchmaker, not a dispatcher**: it makes two people aware of each other; the physical pickup is negotiated and completed off-app. The system has two independent matching mechanisms — a stateless **broadcast plane** ("I have waste", heard by every nearby online collector) and a stateful **request plane** (a household picks one collector and the request goes through a real accept/reject/timeout state machine) — plus a **trust layer** (two-sided ratings with AI-assisted moderation) and an **admin portal** for operations and content moderation.

This SRS covers the **web-only v1** that was actually built and deployed for this examination. A larger design (`borla-technical-design.md` , retained in the repository as a reference) covers a full production system with a native Android collector app, managed Redis/BullMQ, a real SMS gateway, and a separate admin application; §1.4 explains why that design was intentionally descoped, and the full delta is tracked in `Technical_Debt_Plan.md` .

1.3 Definitions

Term	Meaning
Broadcast / pin	A household's "I have waste" announcement, visible to nearby online collectors until cleared or expired
Request	A household's direct ask to one specific collector, with a real accept/reject/timeout lifecycle
Presence	Whether a collector is currently "online" and matchable
Reveal-on-accept	Phone numbers stay hidden until a request is accepted
Double-blind release	A review stays hidden until both sides have reviewed (or the review window closes)
UCP	Use Case Points, the effort-estimation technique used in §7

1.4 Why the scope was reduced from the original design

The original design assumed infrastructure and accounts that cannot be provisioned inside an individual, time-boxed examination: a native Capacitor Android build with a background-location plugin (needs an Android toolchain and a physical/emulated device, and produces an installable APK rather than a gradable "Live Application URL"); a paid Ghanaian SMS gateway account; a managed Redis instance for BullMQ; and a second admin web application. Building the full system would also cost roughly 3,800 person-hours by formal estimation (§7) — nowhere near appropriately scoped for this exam. The requirements below are the **Must-Have** slice that *is* fully implemented, deployed, and tested; everything else is deliberately deferred and tracked as technical debt with a resolution plan (`Technical_Debt_Plan.md`).

2. Overall description

2.1 Product perspective

Borla is a self-contained web application: a single React single-page app (role-routed after login) talking to a single Node/Express API over REST and Socket.IO, backed by one PostgreSQL+PostGIS database. It replaces the two-app, multi-service architecture of the original design with one deployable unit (see `Project_Documentation.md` §9 for the architecture diagram and the explicit list of substitutions).

2.2 User classes

Class	Description	Technical literacy assumed
Household	Has waste to dispose of; broadcasts or requests a pickup	Low–medium; icon-first UI, minimal text entry
Collector	Roams looking for waste to collect for a living	Low–medium; large tap targets, one-tap actions
Admin	Operations/moderation staff	Medium–high; desk user, data-dense screens acceptable

2.3 Operating environment

- **Client**: any modern mobile or desktop browser with Geolocation API support (Chrome, Edge, Firefox, Safari — including Android WebView browsers, since no native app is required for this build).
- **Server**: Node.js 20+, PostgreSQL 16 with the PostGIS extension, deployed on Render.
- **Network**: designed to tolerate patchy/metered mobile connections (see NFR-5).

2.4 Assumptions and dependencies

- Users have a phone number and access to a smartphone/browser; no email is required.
- The OTP is delivered via a real SMS gateway (GiantSMS) when configured — confirmed live in production, the gateway accepts the send request end-to-end; if a send ever fails or no gateway is configured, the OTP is returned in the API response and shown in-app instead as a safe fallback (tracked as TD-02, scheduled, in the Technical Debt Plan, pending visual confirmation on a real handset).
- AI moderation is optional infrastructure: the system is fully functional with `GEMINI_API_KEY` unset, falling back to a manual admin-only moderation queue.
- A single deployed Node process is assumed (no horizontal scaling) — acceptable at pilot scale.

2.5 Constraints

- 48-hour-equivalent time-box for the examination → aggressive MoSCoW prioritisation (§6).
- No budget for paid third-party services (SMS gateway, managed Redis, Vertex AI tier).
- Deployment target restricted to free-tier hosting (Render).

3. Actors

Actor	Type (UCP classification)	Interacts via
Household	Complex (human, GUI)	Web app
Collector	Complex (human, GUI)	Web app
Admin	Complex (human, GUI)	Web app (`/admin`)
Gemini moderation API	Simple (external API)	Server-to-server HTTPS
Scheduler (cron sweeps)	Simple (time-triggered)	In-process, drives system use cases

---

4. Functional requirements

Each requirement is tagged with its MoSCoW priority (§6) and the module that implements it.

ID	Requirement	Priority	Implemented in
FR-01	A user registers/logs in with a phone number and a 6-digit one-time code	Must	`server/src/modules/auth`
FR-02	An admin logs in with a phone + password instead of OTP	Must	`server/src/modules/auth`
FR-03	A household creates a broadcast pin (location, optional waste type, optional note)	Must	`server/src/modules/broadcasts`
FR-04	The system fans the broadcast out to nearby, online, verified, waste-type-matching, non-quiet-hours collectors	Must	`broadcasts/routes.ts` fan-out gauntlet
FR-05	A household clears its own active pin at any time	Must	`broadcasts/routes.ts`
FR-06	A broadcast pin auto-expires after a configurable TTL (default 45 min)	Must	`jobs/index.ts` pin-expiry sweep
FR-07	A collector toggles online/offline and sends periodic heartbeats while online	Must	`server/src/modules/presence`
FR-08	A collector who has been offline (verified) cannot go online (KYC gate)	Must	`presence/routes.ts`
FR-09	Stale presence (no heartbeat for 90s) self-heals to offline	Must	`jobs/index.ts` presence sweep
FR-10	A household views nearby online collectors on a map	Must	`GET /collectors/nearby`
FR-11	A collector views nearby active pins on a map	Must	`GET /pins/nearby`
FR-12	A household sends a direct request to one specific online collector	Must	`server/src/modules/requests`
FR-13	A request moves `requested → seen` when the collector's client opens it	Must	`requests/routes.ts`

ID	Requirement	Priority	Implemented in
FR-14	A collector accepts or rejects a request; the transition is idempotent (a stale reject/accept after resolution is a no-op, not an error)	Must	`requests/routes.ts` conditional `WHERE status IN (...)` updates
FR-15	A request auto-times-out if the collector doesn't respond within a configurable window (default 90s)	Must	`jobs/index.ts` request-timeout sweep
FR-16	Phone numbers stay hidden until a request is accepted, then both sides can see and call each other	Must	`requests/routes.ts` `contact_revealed_at`
FR-17	A household confirms whether a collector actually came for a broadcast pickup ("Did they come?")	Should	`POST /confirmations`
FR-18	Either side of a resolved interaction (accepted request, or a confirmed broadcast) can leave a 1–5 rating + comment for the other	Must	`server/src/modules/reviews`
FR-19	A review/reply is screened (AI if configured, else queued for manual admin approval) before it can go public	Must	`server/src/ai/moderation.ts`
FR-20	A review is only shown once both sides have reviewed, or the review window has closed (double-blind release)	Must	`jobs/index.ts` review-release sweep
FR-21	The reviewed party may post exactly one public reply	Must	`reviews/routes.ts`
FR-22	Any user can report a review for moderation	Should	`POST /reviews/:id/report`
FR-23	An admin can verify a collector, and suspend/reinstate any user	Must	`server/src/modules/admin`
FR-24	An admin can view a moderation queue (AI-flagged + user-reported + awaiting-manual) and resolve items	Must	`admin/routes.ts`
FR-25	An admin can view live stats, a live ops map, and the full audit log of privileged actions	Must	`admin/routes.ts`
FR-26	An admin can retune operational config (pin TTL, radius, timeouts, review window) without a redeploy	Should	`admin/routes.ts` `app_config`

5. Non-functional requirements

ID	Requirement	Approach taken
NFR-1 (Security)	Passwords/OTPs never stored in plaintext; every privileged/authenticated route checks a signed JWT	bcrypt hashing; `requireAuth` / `requireRole` middleware
NFR-2 (Security)	No SQL injection surface	100% parameterised queries (`pg` placeholders), no string-concatenated SQL with user input
NFR-3 (Authorization)	A user can only act within their role and only on their own resources	Role middleware + ownership checks (`WHERE household_id = $1` , etc.) on every mutating route
NFR-4 (Privacy)	A household's contact stays hidden from a specific collector until that collector is chosen and accepts	`contact_revealed_at` ; phone fields stripped from the API response pre-accept

ID	Requirement	Approach taken
NFR-5 (Reliability under poor connectivity)	The app tolerates dropped connections and stale devices	Presence self-heals via TTL-style sweep; idempotent state transitions; polling fallback alongside sockets
NFR-6 (Usability)	Icon-first, large tap targets, minimal required text entry, mobile-first	Design tokens and component library lifted from the approved `borla_UI_design.html`
NFR-7 (Availability)	The deployed instance stays reachable for grading	Render health check (`/api/health`) wired into `render.yaml`
NFR-8 (Data integrity)	A review can never be posted about a fabricated interaction	DB-level `UNIQUE(author_id, request_id)` / `UNIQUE(author_id, broadcast_id)` plus application-level interaction checks
NFR-9 (Fail-safe moderation)	Unmoderated content never goes public by default	Fail-closed: no verdict (missing key, timeout, error) ⇒ stays hidden in the manual queue
NFR-10 (Testability)	Core business logic is covered by automated tests	31 server tests (unit + Supertest integration) + 4 client component tests, all passing — see `Testing_Report.md`

6. Requirement prioritisation (MoSCoW)

Must-have (built, this submission): FR-01–FR-16, FR-18–FR-21, FR-23–FR-25 — the complete two-plane matching engine, masked contact, the full review/moderation/double-blind pipeline, and the admin console.

Should-have (built, lighter-touch): FR-17 (Did-they-come confirmation, minimal UI), FR-22 (user reporting), FR-26 (live config tuning) — all present, but not stress-tested to the same depth as Must-Have items.

Could-have (explicitly deferred — see `Technical_Debt_Plan.md`): native background geolocation, real SMS OTP, Redis/BullMQ + Socket.IO Redis adapter, provider call-masking, photo → waste-type AI classification, admin 2FA, notification rate-limiting nuance beyond a coarse per-IP limit, i18n beyond English.

Won't-have (out of scope entirely, per the original design's own roadmap): in-app payments/escrow, live turn-by-turn tracking, auto-fallback dispatch, multi-neighbourhood expansion.

7. Software effort estimation

Technique chosen: Use Case Points (UCP), cross-checked with expert (top-down) estimation. UCP was chosen over Function Points because the requirements were captured as use cases from the outset (not data-flow specifications), and over raw story points because a first-time solo estimate benefits from UCP's more mechanical, less "gut-feel" weighting — useful for a graded exercise where the reasoning must be shown, not just a number.

7.1 Actor weighting (UAW)

Actor	Classification	Weight
Household	Complex (GUI/human)	3
Collector	Complex (GUI/human)	3
Admin	Complex (GUI/human)	3

Actor	Classification	Weight
Gemini moderation API	Simple (API)	1
Scheduler (cron)	Simple (time-triggered)	1
UAW total		**11**

7.2 Use-case weighting (UUCW)

26 use cases from §4/§6 were classified by transaction count: **4 Complex** (15 pts) — fan-out broadcast creation, request accept/reject with idempotency + reveal, review submission with interaction validation, AI/manual moderation; **8 Average** (10 pts) — OTP login, direct request creation, auto-timeout, auto-expiry, double-blind release, reply, admin verify/suspend, admin moderation resolution; **13 Simple** (5 pts) — the remaining CRUD/toggle use cases (profile edits, presence toggle/heartbeat, nearby queries, pin clear, admin stats/config, reporting, pickup confirmation).

UUCW = (4 × 15) + (8 × 10) + (13 × 5) = 60 + 80 + 65 = **205**

UUCP = UAW + UUCW = 11 + 205 = 216

7.3 Technical Complexity Factor (TCF)

Thirteen factors (T1–T13) rated 0–5 on relevance to Borla (distributed client/server, moderate performance needs, high end-user-efficiency bar for low-literacy users, real geospatial + state-machine logic, single-instance deployment limiting concurrency, strong auth but no 2FA, light third-party access via one AI API). Sum of (weight × rating) = 45.

TCF = 0.6 + (0.01 × 45) = 1.05

7.4 Environmental Factor (EF)

Eight factors (F1–F8) reflecting a solo, AI-assisted, moderately experienced developer with fixed requirements post-planning but no dedicated QA/analyst roles. Sum of (weight × rating) = 18.5.

EF = 1.4 − (0.03 × 18.5) = 0.845

7.5 Adjusted UCP and effort

Adjusted UCP = UUCP × TCF × EF = 216 × 1.05 × 0.845 ≈ 191.6

At the standard **20 person-hours per UCP** (appropriate given no more than two environmental factors were rated unfavourably): **≈ 3,833 person-hours (≈ 22.6 person-months, ≈ 479 person-days)** to build this exact scope to full commercial rigour — complete automated test coverage, polished UX across languages, hardened security, and production-grade infrastructure.

7.6 Expert (top-down) cross-check

A module-by-module expert estimate gives a second, independent number: auth/DB setup 40h, presence 20h, broadcast plane 60h, request plane 50h, realtime layer 30h, reviews+moderation 70h, admin portal 60h, AI integration 20h, full frontend (all screens + styling) 150h, testing 80h, deployment/DevOps 30h, documentation 60h = 670h core work, **+15% coordination/overhead ≈ 770 person-hours (≈ 4.5 person-months solo, full-time)**.

7.7 Why two estimates disagree, and how estimation shaped scope

UCP's 3,833h models a fully-tested, fully-documented, production-hardened build at typical team velocity; the 770h expert estimate models one experienced engineer building a lean-but-real MVP without that last mile of polish. Both numbers are **wildly larger than any literal 48-hour window** — which is the point: the exam itself says not to attempt a large commercial system. The estimate's real job here was to force a decision about *how* to spend a tiny fraction of either number. The choice made was **breadth over depth**: implement a complete, working, thin vertical slice of every Must-Have mechanism (auth → both matching planes → trust layer → admin) end-to-end, rather than fully polishing a narrower subset. The resulting shortfall against both estimates — thinner test coverage than "full rigour" would call for, no UI localisation, coarse rate-limiting, no native mobile client — is exactly what `Technical_Debt_Plan.md` catalogues and schedules for later phases.

7.8 Constraints and assumptions behind the estimate

Assumes: use cases as scoped in §4/§6 (not the original full design); a single developer; PostgreSQL/PostGIS and Render already known technologies (reflected in EF); no requirements churn after the planning phase. Does **not** assume: a dedicated QA engineer, a UX designer beyond the supplied mockup, or a native-mobile specialist — all three would be needed before this UCP estimate could be met in full, and their absence is itself tracked as technical debt.

8. External interface requirements

8.1 REST API (representative — full surface in `Project_Documentation.md` §10)

```
POST /api/auth/otp/request        {phone, role?} -> {devOtp, isNewUser}
POST /api/auth/otp/verify         {phone, code, role?, displayName?} -> {access, refresh, user}
POST /api/auth/admin/login        {phone, password} -> {access, refresh, user}
POST /api/auth/refresh            {refresh} -> {access}
GET  /api/auth/me                 -> {user, profile}

POST /api/presence                {online, lon?, lat?}
POST /api/presence/heartbeat      {lon, lat}
GET  /api/collectors/nearby       ?lon&lat&radius
PATCH /api/households/me          {homeLon?, homeLat?, alertRadiusM?, alertsEnabled?}
PATCH /api/collectors/me          {vehicleType?, wasteTypes?, quietHours?}

POST /api/broadcasts              {lon, lat, wasteType?, note?}
POST /api/broadcasts/:id/clear
GET  /api/broadcasts/me/active
GET  /api/pins/nearby             ?lon&lat&radius
POST /api/confirmations           {broadcastId, came, collectorId?}

POST /api/requests                {collectorId, lon, lat, wasteType?, note?}
GET  /api/requests/mine
GET  /api/requests/:id
POST /api/requests/:id/seen
POST /api/requests/:id/accept
POST /api/requests/:id/reject

POST /api/reviews                 {subjectId, requestId?|broadcastId?, rating, comment?}
GET  /api/users/:id/reviews
POST /api/reviews/:id/reply       {body}
POST /api/reviews/:id/report      {reason}

GET/POST  /api/admin/*            stats, live-map, users, moderation queue, audit log, config
```

8.2 Realtime events (Socket.IO, JWT-authed, room `user:{id}`)

`broadcast:new`, `broadcast:cleared`, `request:new`, `request:seen`, `request:accepted`, `request:rejected`, `request:timed_out`.

---

9. Data requirements

See `Project_Documentation.md` §9.2 for the full ER diagram; the schema itself lives at `server/migrations/001_init.sql` and is the single source of truth.